



Работа с массивами

Время изучения: 14 мин

Автор: Матвей Ефимов, Разработчик учебных программ и эксперт в Go, Python, SQL

Вы изучите:

- [Определение массива](#)
- [Объявление массива](#)
- [Инициализацию массива](#)
- [Индексирование и изменение элементов](#)
- [Длину массива](#)
- [Итерацию по массиву](#)
- [Практические примеры работы с массивами](#)



Определение массива

В языке Go **массив** — это структура данных, которая позволяет хранить фиксированное количество значений **одного и того же типа**. Это как ящик с пронумерованными отделениями: каждое отделение (элемент массива) можно заполнить, изменить или прочесть по номеру (индексу).

Массивы полезны, когда заранее известно, сколько именно значений нужно хранить. Например: 7 дней недели, 12 месяцев, 3 координаты точки.

Что важно помнить:

- **Длина массива фиксирована:** нельзя добавить или удалить элементы после создания.
- **Все элементы одного типа:** например, массив из целых чисел (int) или строк (string), но не смесь.
- **Индексация начинается с нуля:** первый элемент имеет индекс 0, второй — 1, и так далее.

Объявление массива

В Go объявление массива начинается с указания его **размера**, затем **типа данных**, которые он будет содержать. Сначала задаётся длина массива в квадратных скобках, потом тип, затем имя переменной.

Пример: объявление массива из 5 целых чисел

```
var numbers [5]int // Объявляется массив из 5 элементов типа int
```

Разберём этот пример построчно:

- `var` — ключевое слово, обозначающее объявление переменной;
- `numbers` — имя переменной, в которой будет храниться массив;
- `[5]` — длина массива: он будет содержать ровно 5 элементов;
- `int` — тип данных: все элементы будут целыми числами.

В этот момент массив уже существует в памяти. Все 5 его элементов автоматически проинициализированы значением 0 (ноль — это значение по умолчанию для типа `int` в Go).

Альтернативный пример: массив строк

```
var days [7]string // Объявляется массив из 7 строк
```

Пояснение:

- `days` — имя массива;
- `[7]` — он рассчитан на 7 элементов;
- `string` — тип: массив будет хранить строки.

Объявление массива ещё не означает, что он содержит полезные данные. Пока в него не записаны значения, он заполнен значениями по умолчанию:

- 0 для чисел;
- `""` (пустая строка) для строк;
- `false` для булевых значений.

Инициализация массивов

После объявления массива в Go его можно **инициализировать** — то есть сразу задать начальные значения элементов. Это можно сделать несколькими способами: при объявлении, с опущенной длиной или частично. Рассмотрим каждый вариант с подробными примерами.

Инициализация при объявлении

```
var numbers = [3]int{10, 20, 30}
```

Пояснение:

- `var numbers` — объявляется переменная `numbers`;
- `[3]int` — массив из трёх элементов типа `int`;
- `{10, 20, 30}` — список значений для инициализации: первый элемент = 10, второй = 20, третий = 30.

Инициализация с опущенной длиной

Если вы сразу указываете значения, Go может **сам определить длину массива**:

```
var numbers = [...]int{5, 15, 25, 35}
```

Пояснение:

- `[...]int` — троеточие (...) говорит компилятору: «определи длину массива автоматически»;
- `{5, 15, 25, 35}` — четыре элемента, значит, длина массива будет равна 4.

Такой подход удобен, если не хочется вручную считать элементы.

Частичная инициализация

Можно задать значения только для некоторых элементов. Остальные будут заполнены значениями по умолчанию.

```
var numbers = [5]int{1: 100, 3: 300}
```

Пояснение:

- `[5]int` — массив из пяти элементов.
- `{1: 100, 3: 300}` — инициализируем элемент с индексом 1 значением 100, и элемент с индексом 3 значением 300.
- Остальные элементы (индексы 0, 2, 4) будут равны 0 (значение по умолчанию для `int`).

Что стоит запомнить:

- Можно инициализировать массив прямо при создании.
- Используйте ..., если не хотите указывать длину вручную.
- Поддерживается адресная инициализация: можно задать значения только для нужных индексов.

Индексирование и изменение элементов

Массив в Go — это упорядоченная коллекция, где каждому элементу соответствует свой **индекс**. Индексация начинается с **0**, то есть:

- первый элемент — индекс 0;
- второй — 1;
- и так далее.

Через индекс можно **получить доступ к элементу массива**, а также **изменить его значение**.

Доступ к элементу по индексу

```
var numbers = [3]int{10, 20, 30}

fmt.Println(numbers[0]) // Выводит: 10
fmt.Println(numbers[2]) // Выводит: 30
```

Пояснение:

- numbers[0] — доступ к первому элементу массива;
- numbers[2] — доступ к третьему элементу массива.

Изменение значения элемента по индексу

```
numbers[1] = 99 // Заменяем второй элемент (индекс 1) на 99

fmt.Println(numbers) // Выводит: [10 99 30]
```

Пояснение:

- numbers[1] = 99 — меняем значение второго элемента на 99.

Ошибка выхода за границы массива

Если обратиться к несуществующему индексу, программа завершится с ошибкой времени выполнения (runtime panic).

```
fmt.Println(numbers[3]) // Ошибка: индекс выходит за границы  
(index out of range)
```

Почему это ошибка:

- В массиве numbers всего **3 элемента**: индексы 0, 1 и 2.
- Обращение к индексу 3 нарушает границы допустимых значений.

Go строго следит за безопасностью памяти. Он не позволит обратиться к элементу за пределами массива.

Что стоит запомнить:

- Индексация массива начинается с 0.
- Можно читать и изменять элементы по индексу.
- Доступ к недопустимому индексу вызывает аварийное завершение программы.

Длина массива

В языке Go узнать длину массива очень просто — для этого используется встроенная функция **len()**.

Эта функция возвращает количество элементов в массиве, независимо от того, были ли они проинициализированы явно или оставлены по умолчанию.

Пример: получение длины массива

```
var numbers = [4]int{5, 10, 15, 20}  
  
fmt.Println(len(numbers)) // Вывод: 4
```

Пояснение по строкам:

- `var numbers = [4]int{5, 10, 15, 20}` — создаём массив длиной 4;
- `len(numbers)` — функция `len` возвращает значение 4, потому что в массиве 4 элемента;
- `fmt.Println(...)` выводит это значение на экран.

Пример: длина массива с неинициализированными элементами

```
var emptyArray [10]string  
  
fmt.Println(len(emptyArray)) // Вывод: 10
```

Пояснение:

- Мы создали массив из 10 строк, но не задали им значения.
- Все элементы будут пустыми строками (""), но len() всё равно вернёт 10, так как длина задаётся при объявлении и не зависит от заполненности.

Что стоит запомнить:

- Функция len() всегда возвращает фиксированную длину массива, которая была задана при его создании.
- Длина массива в Go не меняется после объявления.

Итерация по массиву

Чтобы обработать каждый элемент массива, часто используют **циклы**. В Go это можно сделать двумя основными способами:

1. С помощью классического цикла for с индексом.
2. С помощью цикла for range, который упрощает работу с массивами.

Цикл for с индексом

Полный контроль над индексами и значениями.

```
var numbers = [4]int{10, 20, 30, 40}  
  
for i := 0; i < len(numbers); i++ {  
    fmt.Println("Индекс:", i, "Значение:", numbers[i])  
}
```

Пояснение по строкам:

- var numbers = [4]int{10, 20, 30, 40} — создаём массив из 4 целых чисел;
- for i := 0; i < len(numbers); i++ — цикл начинается с i = 0 и продолжается, пока i меньше длины массива;

- `numbers[i]` — доступ к элементу массива по текущему индексу;
- `fmt.Println(...)` — выводим индекс и значение элемента.

Цикл `for range`

Упрощённая форма цикла, которая сразу даёт и индекс, и значение.

```
var numbers = [4]int{10, 20, 30, 40}

for index, value := range numbers {
    fmt.Println("Индекс:", index, "Значение:", value)
}
```

Пояснение по строкам:

- `range numbers` — позволяет пройти по всем элементам массива;
- `index` — текущий индекс в массиве;
- `value` — значение элемента с этим индексом;
- `fmt.Println(...)` — печатаем оба значения.

Если индекс не нужен, его можно пропустить с помощью подчёркивания (`_`):

```
for _, value := range numbers {
    fmt.Println("Значение:", value)
}
```

Что стоит запомнить:

- Классический `for` даёт больше контроля, например, если нужно пройти не по всем элементам или в обратном порядке.
- `for range` — короткий и читаемый способ пройти по всем элементам массива.
- В обоих случаях можно использовать `len()` для получения длины массива.

Практические примеры

В этом разделе покажем, как использовать массивы в простых, но полезных задачах: **заполнение значениями, поиск элемента, копирование массива.**

Пример 1: Заполнение массива значениями

Заполним массив квадратами чисел от 1 до 5.

```
var squares [5]int // Создаём массив из 5 целых чисел

for i := 0; i < len(squares); i++ {
    squares[i] = (i + 1) * (i + 1) // Записываем квадрат числа
    (индекс + 1)
}

fmt.Println(squares) // Вывод: [1 4 9 16 25]
```

Пояснение:

- $squares[i] = (i + 1) * (i + 1)$ — так как индексация с нуля, прибавляем 1 для получения правильного числа;
- В результате массив содержит: $1^2, 2^2, \dots, 5^2$.

Пример 2: Поиск элемента в массиве

Ищем, есть ли в массиве заданное число.

```
var numbers = [5]int{3, 7, 1, 9, 5}
var target = 9
var found = false

for i := 0; i < len(numbers); i++ {
    if numbers[i] == target {
        fmt.Println("Найдено на позиции:", i)
        found = true
        break // Прерываем цикл после нахождения
    }
}

if !found {
    fmt.Println("Элемент не найден")
}
```


Пояснение:

- `target` — искомое значение;
- `if numbers[i] == target` — сравниваем каждый элемент;
- `found` — логическая переменная, чтобы отслеживать результат;
- `break` — останавливаем цикл, если элемент найден.

Пример 3: Копирование одного массива в другой

Создадим копию массива вручную.

```
var original = [3]string{"Go", "Java", "Python"}
var copy [3]string // Массив такой же длины

for i := 0; i < len(original); i++ {
    copy[i] = original[i] // Копируем каждый элемент
}

fmt.Println("Исходный:", original)
fmt.Println("Копия    :", copy)
```

Пояснение:

- `original` — исходный массив строк;
- `copy` — новый массив той же длины;
- Проходим по каждому элементу и копируем его вручную.

В Go также можно просто присвоить один массив другому:

`copy = original`, но в обучающих целях лучше разобрать копирование вручную.

Что нужно запомнить:

- Массив можно заполнять значениями через цикл `for`.
- Для поиска значения используйте цикл и условие `if`.
- Массив можно копировать вручную или с помощью прямого присваивания.

Подведём итоги

- **Массив** — это набор элементов фиксированной длины.
- **Все элементы одного типа** (например, int, string, bool).
- Индексация начинается **с нуля**.
- Массив можно **объявлять, инициализировать, изменять и перебирать**.
- Используйте функцию len() для получения длины.
- Циклы for и for range удобны для итерации.

Когда использовать массивы:

- **Фиксированная длина известна заранее.** Например, вы точно знаете, что нужно 12 ячеек — по одной на каждый месяц года.
- **Нужна максимальная производительность.** Массивы в Go размещаются в памяти как единый блок, что может быть быстрее, чем использование слайсов.
- **Предпочтение статической структуре.** Если структура не изменяется, массив проще в использовании и понимании, чем слайс.

Но если размер может изменяться — лучше использовать **срезы (slices)**, они гибче.

Полезные ссылки:

- [Официальная документация по массивам \(Go Tour\)](#)
- [Package builtin — функция len](#)
- [Effective Go — Arrays](#)

Мини-задача для практики:

Создайте массив из 7 целых чисел. Заполните его значениями от 1 до 7, но при этом умножьте каждое значение на 10. Затем выведите элементы массива на экран в обратном порядке.

Что попрактикуете:

- объявление массива;
- заполнение значениями в цикле;
- обратный проход по массиву через for.